

Del código al diagrama

Se abre con la solución de la actividad, y de ahí en adelante lo mismo que ya está escrito se representa en UML: la caja, las relaciones y la multiplicidad.

Rubén Schnettler • Docente

INACAP Valparaíso • Analista Programador

BLOQUE 1 • SEGUNDA PARTE

REVISIÓN EN PLENARIO

Una solución posible

gimnasio.py

```
class Socio:
    def __init__(self, nombre, numero, al_dia):
        self.nombre = nombre
        self.numero = numero
        self.al_dia = al_dia
        self.visitas = 0

    def registrar_visita(self):
        self.visitas += 1
        return self.visitas

    def puede_entrar(self):
        return self.al_dia
```



LA DECISIÓN INTERESANTE

visit no aparece entre los parámetros: un socio nuevo siempre empieza en cero, así que el valor se fija dentro del constructor.



Y EL SEGUNDO MÉTODO

Devuelve el atributo directamente. Un método puede ser de una sola línea y estar perfectamente bien.

REVISIÓN EN PLENARIO

La consulta, y la salida

```
socios = [  
    Socio("Carla Rojas", 1001, True),  
    Socio("Diego Salas", 1002, False),  
    Socio("Ana Pérez", 1003, True),  
]  
  
print("Pueden entrar hoy:")  
  
for s in socios:  
    if s.puede_entrar():  
        print(" -", s.nombre)
```

Terminal

```
$ python gimnasio.py  
Pueden entrar hoy:  
- Carla Rojas  
- Ana Pérez
```



CON ESTO SE CIERRA LA CLASE

Se escribió una clase, se crearon objetos, se les hizo una pregunta y el programa respondió. **Eso es programación orientada a objetos**, y el resto de la unidad es lo mismo a mayor escala.

SEGUNDA PARTE

DEL CÓDIGO AL DIAGRAMA

Dibujar lo escrito

El mismo código que se acaba de escribir, ahora representado en un diagrama de clases UML.

RECUPERACIÓN

Dónde quedamos



LO CUBIERTO HASTA AQUÍ

Clase, objeto, atributo y método. Y qué introdujo la POO respecto del código clásico.



LO QUE YA SE HIZO

La clase **Socio** escrita y ejecutando, con dos objetos.



LO QUE FALTA

Poder **mostrar ese diseño sin mostrar el código**. Para eso existe UML.



POR QUÉ IMPORTA

En un equipo nadie discute el diseño leyendo código. Se discute **mirando un diagrama**, porque se abarca en una sola lectura.

BLOQUE

05

La notación UML

Cómo se dibuja una clase, y qué significa
cada símbolo

CONCEPTO

Qué es UML y para qué sirve



UML • LENGUAJE UNIFICADO DE MODELADO

Una notación **estándar** para representar la estructura de un sistema. Su valor está en que cualquier desarrollador la entiende sin explicación previa.



POR QUÉ EL ESTÁNDAR

Un diagrama inventado obliga a explicarlo. Un diagrama UML se lee solo.



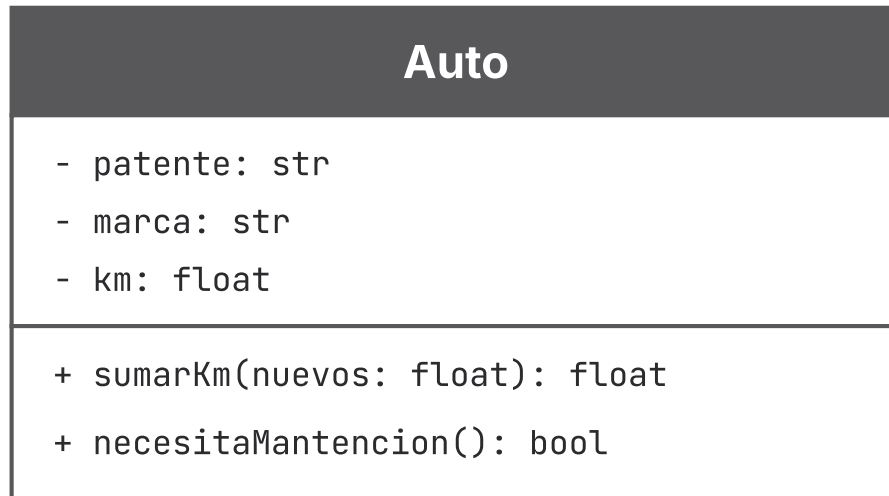
LA VERSIÓN

UML 2.5.1. En internet circulan ejemplos con notaciones antiguas o simplemente incorrectas.

De los catorce tipos de diagrama que define UML, en esta unidad se usa uno solo: el **diagrama de clases**.

NOTACIÓN • LO BÁSICO

La caja de una clase, piso por piso

**PISO 1 • EL NOMBRE DE LA CLASE**

Una sola palabra, en singular y con mayúscula inicial.

PISO 2 • LOS ATRIBUTOS

El signo, el nombre del dato y, tras los dos puntos, el tipo: texto, entero, decimal o verdadero/falso.

PISO 3 • LOS MÉTODOS

El signo, el nombre, los paréntesis con lo que recibe y, al final, qué devuelve.

Es la misma información del archivo `taller.py`, ordenada en tres pisos. **No hay contenido nuevo que aprender: hay otro formato para lo mismo.**

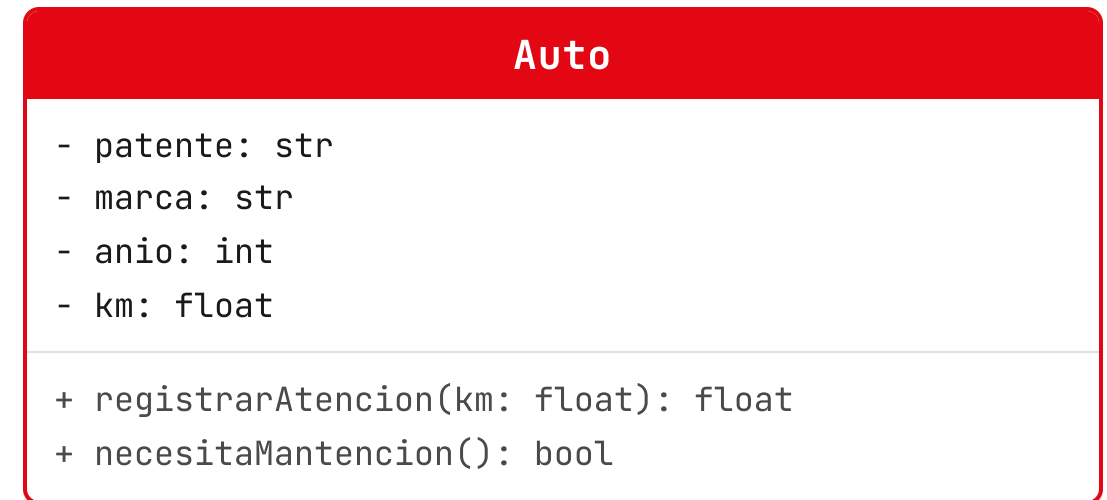
NOTACIÓN 1 DE 3

De su código a la caja

Lo que se escribió antes

```
class Auto:
    def __init__(self, patente, marca, anio, km):
        self.patente = patente
        ...
    def registrar_atencion(self, km_nuevos):
        ...
    def necesita_mantencion(self):
        ...
```

La misma clase, en UML



Arriba el nombre • al medio los atributos **con su tipo** • abajo los métodos con parámetros y retorno.

EJERCICIO 4 • INDIVIDUAL

6 minutos

Leer una caja antes de dibujar una

Libro

```
- isbn: str
- titulo: str
- copiasDisponibles: int

+ prestar(rut: str): bool
+ devolver(rut: str): void
```

De una biblioteca.

Responda las cinco preguntas mirando solo la caja. **No hace falta ver el código.** Ese es justamente el punto.

- 1 ¿Cómo se llama la clase, y en qué piso está escrito su nombre?
- 2 ¿Cuántos atributos tiene, y de qué tipo es `copiasDisponibles`?
- 3 ¿Qué significa el signo `-` delante de cada atributo?
- 4 ¿Qué recibe y qué responde el método `prestar()`?
- 5 ¿Qué quiere decir `void` al final de `devolver()`?

RESPUESTA • EJERCICIO 4

Revisión en plenario

Lo que la caja dice, sin abrir el código

Libro

```
- isbn: str
- titulo: str
- copiasDisponibles: int

+ prestar(rut: str): bool
+ devolver(rut: str): void
```

- 1 **Libro**, en el piso de arriba. En singular y con mayúscula inicial, como manda la convención.
- 2 **Tres**. `copiasDisponibles` es `int`: un número entero, sin decimales. No existe media copia.
- 3 **Privado**. Solo la propia clase puede modificarlo. En Python se escribe con dos guiones bajos: `self.__isbn`.
- 4 Recibe el **RUT de quien pide el libro**, que es texto, y responde `bool`: verdadero si el préstamo se pudo hacer, falso si no quedaban copias.
- 5 **Que no devuelve nada**. Hace su trabajo y termina. En Python es un método sin `return`.

NOTACIÓN 2 DE 3

Visibilidad: el símbolo antes del nombre

SÍMBOLO	VISIBILIDAD	QUIÉN PUEDE USARLO	CÓMO SE ESCRIBE EN PYTHON
-	Privado	Solo la propia clase	<code>self.__salario</code> · dos guiones bajos delante
+	Público	Cualquier clase del sistema	<code>self.nombre</code> · sin nada delante
#	Protegido	La clase y sus herederas	<code>self._area</code> · un solo guion bajo



REGLA PRÁCTICA

Atributos privados, métodos públicos. Así se representa el encapsulamiento: el dato protegido y el acceso controlado. Es lo que se espera en una asignatura con «seguro» en el nombre.

NOTACIÓN 3 DE 3

La firma completa de un método

```
+ registrarAtencion(kmNuevos: float): float
```

Diagrama de anotación de la firma de método:

- visibilidad: `+`
- nombre del método: `registrarAtencion`
- parámetros, con su tipo: `(kmNuevos: float)`
- qué devuelve: `: float`

**FIRMA INCOMPLETA**

registrarAtencion() · no dice qué recibe ni qué responde.

**FIRMA COMPLETA**

Con parámetros y retorno se sabe cómo usar el método **sin abrir el código**.

EJERCICIO 5 • INDIVIDUAL

8 minutos

Del código a la caja, en el cuaderno

```
class Producto:
    def __init__(self, codigo, nombre, precio, stock):
        self.__codigo = codigo
        self.__nombre = nombre
        self.__precio = precio
        self.__stock = stock

    def vender(self, cantidad):
        # descuento del stock y responde True o False
        ...

    def valor_inventario(self):
        return self.__precio * self.__stock
```



Dibuje la caja

- Los tres pisos
- El signo y el tipo de cada atributo
- Los parámetros de cada método
- Qué devuelve cada método

Dos avisos: el tipo **no está escrito en Python**, se deduce del dato; y los nombres de método **cambian de forma** al pasar al diagrama.

RESPUESTA · EJERCICIO 5

Revisión en plenario

La caja de Producto, resuelta

Producto

```
- codigo: str
- nombre: str
- precio: float
- stock: int

+ vender(cantidad: int): bool
+ valorInventario(): float
```

- 1 Los **dos guiones bajos** de `self.__precio` significan privado: en la caja se escriben con `-`.
- 2 Los tipos salen **del sentido del dato**: un código es texto (`str`), un precio lleva decimales (`float`), un stock se cuenta en unidades enteras (`int`).
- 3 `valor_inventario` pasa a `valorInventario`: en el diagrama se usa camelCase, en Python snake_case. **Ninguna de las dos está mal.**
- 4 El constructor `__init__` **no se dibuja** como método: lo que hace es dejar los atributos del piso del medio.

LA HERRAMIENTA

diagrams.net, antes conocida como draw.io

Se abre en el navegador desde **app.diagrams.net**. No requiere instalación ni cuenta de pago.



POR QUÉ ESTA

Tiene una biblioteca UML completa: los rombos y triángulos ya vienen con la forma correcta del estándar.



LO QUE NO SIRVE

Una foto de una pizarra o un dibujo en un procesador de texto.



Alternativas válidas

- StarUML
- Visual Paradigm
- PlantUML, si prefiere escribir texto

El nombre draw.io sigue funcionando y redirige al mismo sitio.

HÁGALO AHORA

Configurar diagrams.net para UML

1 Abrir **app.diagrams.net** y activar la biblioteca UML

Plantilla **Blank Diagram**. Luego **More Shapes** → marcar **UML** → **Apply**.

2 Arrastrar la forma «Class» al lienzo

Doble clic sobre cada zona para escribir nombre, atributos y métodos.

3 Unir clases desde el borde

Al acercar el cursor al borde aparecen flechas azules: desde ahí se arrastra la línea.

4 Exportar para el informe

File → **Export as** → **PNG**, con **Zoom 200 %**. Con el zoom por omisión la imagen sale borrosa.

ACTIVIDAD 1 • INDIVIDUAL

20 minutos

Dibujar la clase que usted escribió

1. Abrir el archivo `gimnasio.py` que usted escribió
2. Dibujar en diagrams.net la clase `Socio`
3. Atributos privados -, cada uno con su tipo
4. Métodos públicos +, con parámetros y retorno
5. Comparar el diagrama con el código: deben decir lo mismo

Es la misma información en dos formatos. Si algo está en el código y no en el diagrama, el diagrama está incompleto.



RECORDAR LA CONVENCION

En el código va `registrar_visita` ;
en el diagrama, `registrarVisita` .

RESPUESTA • ACTIVIDAD 1

Revisión en plenario

La clase Socio, dibujada

Socio

```
- nombre: str
- numero: int
- alDia: bool
- visitas: int

+ registrarVisita(): int
+ puedeEntrar(): bool
```

- 1 **Cuatro atributos, no tres.** `visitas` no aparece entre los paréntesis del constructor porque parte en cero, pero es un dato que el objeto guarda: va en la caja igual que los demás.
- 2 `alDia` es `bool`: solo puede valer verdadero o falso.
- 3 `registrarVisita()` no recibe nada —los paréntesis van vacíos— y devuelve el total de visitas después de sumar la nueva.
- 4 Los dos métodos son **públicos**: son la puerta de entrada al objeto. Los atributos son **privados**. Eso es el encapsulamiento, dibujado.

BLOQUE

06

Las clases se relacionan

Una clase sola no es un sistema

EL PROBLEMA

Una clase sola alcanza para muy poco

La clase **Auto** resuelve un auto. Pero el taller necesita saber **de quién es cada auto y qué atenciones tuvo**. Eso ya no cabe en una sola caja.

**Auto**

patente, marca, año, kilómetros

**Cliente**

nombre, teléfono, correo

**Atencion**

fecha, descripción, costo

**LA PREGUNTA QUE SIGUE**

Tres cajas sueltas tampoco son un sistema. **Falta decir cómo se vinculan**, y qué pasa con una si desaparece la otra. Eso son las relaciones.

ANTES DEL DETALLE

Las cuatro relaciones, en una lámina

N.º	SÍMBOLO	NOMBRE	SE LEE	LA PREGUNTA QUE LA DECIDE	EJEMPLO
R1		Asociación	«se relaciona con»	¿Las dos existen por separado?	Cliente y Auto
R2		Agregación	«agrupa a»	¿La parte sobrevive sin el todo? Sí	Taller y Mecanico
R3		Composición	«está compuesto por»	¿La parte sobrevive sin el todo? No	Auto y Atencion
R4		Generalización	«es un»	¿Se puede decir «X es un Y»?	Camion y Vehiculo

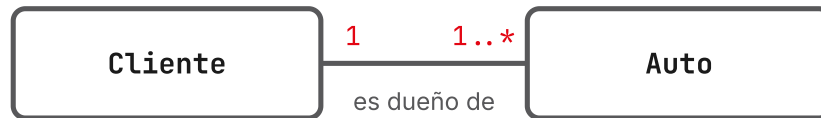


QUÉ PASA SI EL SÍMBOLO MIRA AL OTRO LADO

El rombo y el triángulo **tienen lado**: girarlos **afirma lo contrario**. Un rombo junto a Mecánico diría que el mecánico agrupa talleres. La **R1** es la única sin lado.

R1 • ASOCIACIÓN

«Se relaciona con»



Línea simple, sin adornos.

Dos clases **se conocen y colaboran**, pero cada una existe por su cuenta.

La prueba: si el cliente se va del taller, el auto sigue existiendo, y al revés.

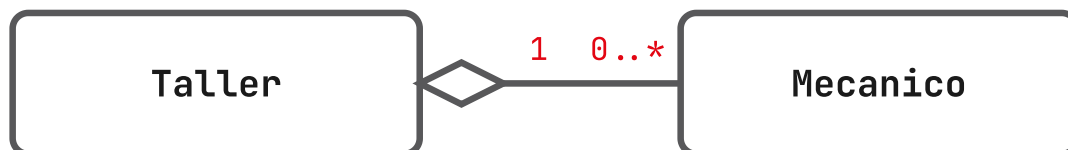
#

LOS NÚMEROS PEQUEÑOS DE LOS EXTREMOS

Son la **multiplicidad**: cuántos objetos hay de cada lado. Se lee «un cliente es dueño de uno o muchos autos; cada auto tiene un solo dueño».

R2 Y R3 · AGREGACIÓN Y COMPOSICIÓN

«Tiene un», con dos intensidades

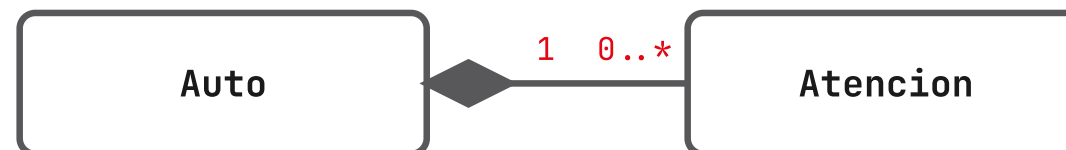


Agregación · rombo **vacío**. El taller agrupa mecánicos, pero si el taller cierra, los mecánicos siguen existiendo: se van a otro.



LA PREGUNTA QUE DECIDE

¿La parte sobrevive sin el todo? Si sobrevive, es agregación. Si no, es composición.



Composición · rombo **lleno**. Una atención no existe sin su auto: si se borra el auto, sus atenciones no significan nada.



EL ROMBO TIENE LADO

Va **siempre junto al todo**. Puesto junto a **Mecanico**, el diagrama diría que el mecánico agrupa talleres.

EJERCICIO 6 • EN PAREJA 10 minutos

¿Qué relación es, y por qué?

Para cada situación, indique **cuál de las cuatro relaciones** corresponde. Se evalúa la justificación, no la palabra.

- 1 Un cliente registra sus autos en el taller. Si el cliente se cambia de taller, los autos siguen existiendo
- 2 Cada atención queda anotada en la ficha del auto. Si el auto se da de baja, sus atenciones no significan nada
- 3 Un gimnasio tiene monitores contratados. Si el gimnasio cierra, los monitores trabajan en otro
- 4 Un camión tiene patente, marca y kilómetros, igual que cualquier vehículo, y además capacidad de carga
- 5 Una boleta se compone de líneas de detalle, y esas líneas no existen fuera de su boleta

RESPUESTA • EJERCICIO 6

Revisión en plenario

Las cinco relaciones, justificadas

N.º	RELACIÓN	LA JUSTIFICACIÓN QUE SE ESPERA
1	Asociación	Se conocen y colaboran, pero cada uno existe por su cuenta
2	Composición	La atención no tiene sentido sin su auto: la parte muere con el todo
3	Agregación	El gimnasio los agrupa, pero el monitor sobrevive al cierre
4	Generalización	«Un camión <i>es un</i> vehículo»: la frase se sostiene
5	Composición	Sin boleta no hay línea de detalle que valga



EL CRITERIO QUE RESUELVE LA DUDA

Pregúntese **qué pasa con la parte si se borra el todo**. Si desaparece, es composición. Si sigue viva, es agregación. Si no hay todo ni parte, es asociación. Y si se puede decir «es un», es generalización.

EL DATO QUE MÁS SE OLVIDA

Multiplicidad: cuántos de cada lado

NOTACIÓN	SE LEE	EN EL TALLER
1	Exactamente uno	Un auto tiene un dueño
0..1	Ninguno o uno	Un auto recién ingresado puede no tener dueño registrado
1..*	Uno o muchos	Un cliente que llega al taller tiene al menos un auto
0..*	Ninguno o muchos	Un auto nuevo puede no tener atenciones todavía



DE DÓNDE SALE CADA NÚMERO

Del **enunciado**, no de la intuición. Cada vez que un enunciado dice «uno solo», «uno o varios» o «puede no tener», está fijando una multiplicidad.



POR QUÉ NO EXISTE 1..0

El rango se escribe **del mínimo al máximo**:
0..1 es «ninguno o uno». 1..0 pediría al menos uno y como máximo ninguno, que es imposible: no es notación válida, es un error.

EJERCICIO 7 • INDIVIDUAL

6 minutos

Poner los números en cada extremo

- 1 Un cliente puede tener varios autos en el taller; cada auto tiene un solo dueño
- 2 Un auto recién ingresado puede no tener ninguna atención; con el tiempo tendrá muchas
- 3 Un socio se inscribe en varias clases dirigidas, y una clase dirigida admite varios socios

Complete los seis extremos que faltan

1) Cliente	----	_____	----	Auto
2) Auto	----	_____	----	Atencion
3) Socio	----	_____	----	ClaseDirigida

Las opciones son cuatro: $1 \cdot 0 \dots 1 \cdot 1 \dots * \cdot 0 \dots *$

RESPUESTA · EJERCICIO 7

Revisión en plenario

Los números, y de qué lado van

1)	Cliente	1	_____	1..*	Auto
2)	Auto	1	_____	0..*	Atencion
3)	Socio	0..*	_____	0..*	ClaseDirigida



DE QUÉ LADO VA EL NÚMERO

«Un cliente tiene uno o muchos autos» pone el 1..* **junto a Auto**, que es lo que se está contando. No junto a Cliente. Es el error más repetido.



POR QUÉ EL CASO 3 LLEVA CERO

Un socio recién inscrito **todavía no eligió ninguna clase**, y una clase nueva parte sin inscritos. El cero se pone cuando el enunciado admite que no haya ninguno.

EL MÉTODO

Cómo se arma un diagrama, en cinco pasos

1 Subraye los sustantivos del enunciado

Cliente, auto, atención: el sustantivo que se repite es candidato a clase.

2 Descarte los que en realidad son datos

«Patente» se repite, pero no hace nada: es atributo de Auto, no una clase.

3 Dibuje las cajas vacías y únalas

Primero la estructura: qué se relaciona con qué, y de qué tipo.

4 Complete el piso del medio

Los atributos, cada uno con su signo de visibilidad y su tipo de dato.

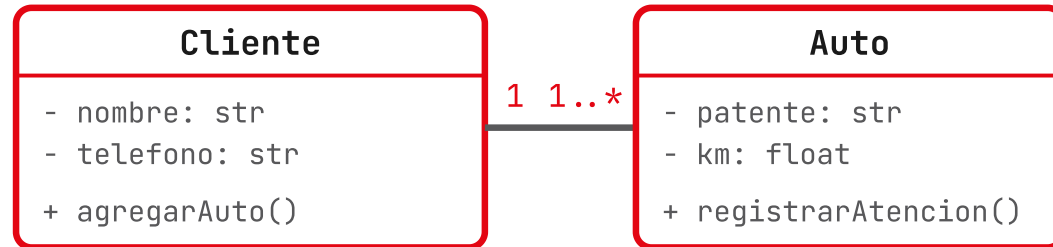
5 Agregue los métodos y la multiplicidad

Lo que cada clase sabe hacer, y cuántos participan en cada extremo.

El orden importa: partir por los atributos deja cajas correctas que no se relacionan con nada.

SE MODELA EN PANTALLA

El taller, con dos clases relacionadas



La relación, en código

```
class Cliente:
    def __init__(self, nombre, telefono):
        self.nombre = nombre
        self.telefono = telefono
        self.autos = []

    def agregar_auto(self, auto):
        self.autos.append(auto)
```

`self.autos = []` La lista vacía **es la relación**: aquí se guardarán los autos del cliente

`append()` *añadir* · agrega un elemento al final de una lista

ACTIVIDAD 2 • INDIVIDUAL

40 minutos

Dos clases relacionadas, dibujadas y escritas



ENUNCIADO AMPLIADO

El gimnasio ahora también registra las **clases dirigidas**: cada clase tiene un nombre, un horario y un cupo máximo. Un socio puede **inscribirse en varias clases**, y una clase puede tener **varios socios inscritos**.



Qué entregar

- El diagrama con **las dos clases**
- El tipo de relación, justificado
- La multiplicidad en los dos extremos
- El código de las dos clases
- Una inscripción hecha y un `print`

REVISIÓN EN PLENARIO

Una solución posible

```
class ClaseDirigida:
    def __init__(self, nombre, horario, cupo):
        self.nombre = nombre
        self.horario = horario
        self.cupo = cupo
        self.inscritos = []

    def inscribir(self, socio):
        if len(self.inscritos) ≥ self.cupo:
            return False
        self.inscritos.append(socio)
        return True
```



LA RELACIÓN ES DE MUCHOS A MUCHOS

0 .. * en los dos extremos, y es una **asociación**: si se cancela una clase, los socios siguen existiendo.

len() *length* · cuántos elementos tiene una lista

if Ejecuta el bloque **solo si** la condición es verdadera

≥ Mayor o igual. Aquí sirve para no pasar del cupo

CIERRE DEL BLOQUE

Leer un diagrama en voz alta

1 Las cajas

«El modelo tiene tres clases: Cliente, Auto y Atencion.»

2 Las relaciones, con su nombre

«Cliente se asocia con Auto; Auto se compone de Atencion.»

3 La multiplicidad, en palabras

«Un cliente tiene uno o muchos autos, y cada auto un solo dueño.»

4 La decisión, justificada

«Es composición porque la atención no existe sin su auto.»



ESTO ES LA DEFENSA ORAL

Cuatro de los veinte indicadores de la rúbrica **se evalúan en la defensa oral**. Lo que se pide es exactamente este recorrido de cuatro pasos sobre el propio diagrama.

Conviene practicarlo en voz alta desde hoy, con el diagrama del gimnasio. Un modelo que no se puede explicar en voz alta todavía no está terminado.

TERCERA PARTE

LA IA EN EL MODELADO

La IA propone. Usted decide

Lo mínimo para cumplir con lo que exige la evaluación: cómo se pide, qué se documenta y cómo se registra la decisión.

LO QUE EXIGE EL CRITERIO 1.1.3

Cuatro evidencias obligatorias

1 Dos iteraciones de propuesta con la herramienta

Cada una con al menos 3 clases, atributos, métodos y relaciones. Una se elige como base y la otra se descarta, con razón.

2 Dos prompts documentados, con su resultado

Con el contexto del problema y las instrucciones de formato. Se anexa lo que devolvió la herramienta.

3 Cuatro elementos analizados

Errores, similitudes o diferencias, clasificados según el aspecto del modelo y contrastados con el análisis propio.

4 El modelo ajustado, con fundamento técnico

Cada modificación explicada con criterios de diseño orientado a objetos.

HERRAMIENTA DE TRABAJO · CLARO

Un prompt útil: el esquema CLARO

- C Contexto** · el enunciado del caso, pegado completo
- L Labor** · «Proponga un diagrama de clases», con un verbo
- A Alcance** · «Máximo 6 clases. No incluya interfaz de usuario»
- R Rol** · «Actúe como arquitecto de software con experiencia en UML»
- O Orden** · «En texto, con atributos tipados y multiplicidad en cada relación»



PROMPT INSUFICIENTE

«hazme un diagrama de clases para una empresa»

Menos de diez palabras. La rúbrica lo ubica en nivel elemental.



PROMPT ADECUADO

Con las cinco letras. Y lo importante: **queda registrado** tal como se escribió, para poder anexarlo al informe.

LA INTERACCIÓN EN LA PRÁCTICA

Del prompt genérico al prompt dirigido



ITERACIÓN 1

Proponga un diagrama de clases para el sistema de gestión interna de EcoTech Solutions.

Resultado: ocho clases, sin tipos de dato, con una clase Sistema que hace de todo y sin multiplicidades.



ITERACIÓN 2

Actúe como arquitecto de software. Con el caso adjunto, proponga máximo 6 clases con atributos tipados, métodos con retorno y multiplicidad en cada relación. No incluya interfaz de usuario.

Resultado: cinco clases con tipos y multiplicidades, aún con errores, pero ya comparable con el modelo propio.

Ambas iteraciones van al informe. La segunda se usa como base; la primera se descarta **explicando por qué**, y eso es lo que exige el nivel experto.

QUÉ REVISAR SIEMPRE

Los errores frecuentes de un modelo generado

ASPECTO	ERROR HABITUAL	CÓMO SE DETECTA
Clases	Inventa una clase Sistema o Gestor que concentra todo	Tiene más de seis métodos y ninguna responsabilidad clara
Clases	Convierte atributos en clases: Salario, Correo	La clase tiene un solo atributo dentro
Atributos	Omite los tipos de dato o los deja en inglés	Se lee nombre sin nada después de los dos puntos
Relaciones	Usa asociación para todo, sin agregación ni composición	Todas las líneas son iguales
Relaciones	Omite la multiplicidad, o pone 1 .. * en ambos extremos	Contrastando con el enunciado del caso

Estos cinco cubren la mayoría de los casos. Encontrar cuatro de ellos ya cumple el indicador 1.1.3.I.13 en nivel experto.

CÓMO SE REGISTRA EL ANÁLISIS

La tabla que va en el informe

#	ASPECTO	TIPO	QUÉ PROPUSO LA IA	QUÉ DECIDÍ Y POR QUÉ
1	Clases	Error	Una clase Sistema con doce métodos	Se elimina: viola responsabilidad única. Sus métodos se reparten
2	Relaciones	Error	Asociación entre Empleado y RegistroTiempo	Se cambia a composición: no existe sin el empleado
3	Atributos	Diferencia	Salario como atributo público	Se deja privado, por el requisito de cifrado
4	Clases	Similitud	Identificó las mismas cuatro entidades	Se conserva: confirma el análisis propio del Paso 1

La última columna es la que se evalúa. Describir lo que propuso la IA es fácil; decidir y fundamentar es lo que distingue el nivel experto, y lo que habrá que defender oralmente.

DEL CÓDIGO AL DIAGRAMA

Primero se entiende, después se dibuja, **al final se codifica**

El método ya está completo en pequeño, y la IA es una herramienta dentro de él. Lo que sigue es aplicarlo al caso de la evaluación.

TI3V21 • Programación Orientada a Objeto Seguro • INACAP Valparaíso